Subject : Programming for Problem Solving Lab.

Unit 4 : Arrays

Program : B.Tech. - CSE

Course Code : **CSCR 1501**

Semester : 1st

Faculty Teaching : Dr. Shree Harsh Attri

Designation : Associate Professor

Department : Computer Sc. & Engineering

Sharda University, Greater Noida, U.P

Arrays

An array is a collection of elements of similar type of datatypes.

```
int x1 = 10;
int x2 = 20;
int x3 = 30;
int x4 = 40;
int x5 = 50;
.
.
.
.
int x10 = 100;

and so on
```

10	20	30	40	50	60	70	80	90	100

Single Array to store all values of similar type

Arrays

1D Array declaration

```
int arr[5];
```

- data type
- name of array
- size of array

	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
Array Elements	100	200	300	400	500
Array Indices	0	1	2	3	4

SHARDA UNIVERSITY
Beyond Boundaries

1D Array declaration and initialization

int arr[5];

data type | name of array | size of array

Syntax:

```
int arr[] = {100,200,300,400,500};

int arr[5] = {100,200};

int arr[5] = {};
```

	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
Array Elements	100	200	300	400	500
Array Indices	0	1	2	3	4
Array Location	642248	642252	642256	642260	642264

100	200	0	0	0

0	0	0	0	0

SHARDA UNIVERSITY
Beyond Boundaries

1D Array initialization and access array elements

```c
#include<stdio.h>
main()
{
    int i, arr[5];
    printf("Enter the elements in the array:\n");
    for(i=0;i<5;i++)
    {
        scanf("%d",&arr[i]);
    }
    // Access array elements using indices
    printf("\n\nEntered elements in an Array\n");
    for(i=0;i<5;i++)
    {
        printf("Elements in an array: %d at location %d\n",arr[i],&arr[i]);
    }
}
```

gb = garbage value

gb	gb	gb	gb	gb
0	1	2	3	4

10	20	30	40	50
0	1	2	3	4
642248	642252	642256	642260	642264

1D Array initialization and access array elements

```c
#include<stdio.h>
main()
{
    int i, arr[5];
    printf("Enter the elements in the array:\n");
    for(i=0;i<5;i++)
    {
        scanf("%d",&arr[i]);
    }
    // Access array elements using indices
    printf("\n\nEntered elements in an Array\n");
    for(i=0;i<5;i++)
    {
        printf("Elements in an array: %d at location %d\n",arr[i],&arr[i]);
    }
}
```

```
Enter the elements in the array:
10
20
30
40
50


Entered elements in an Array
Elements in an array: 10 at location 6422248
Elements in an array: 20 at location 6422252
Elements in an array: 30 at location 6422256
Elements in an array: 40 at location 6422260
Elements in an array: 50 at location 6422264
```

Program to sum the elements of an Array

```c
#include<stdio.h>
main()
{
    int arr[5],i,sum=0;
    printf("Enter the elements in an Array\n");
    for(i=0;i<5;i++)
    {
        scanf("%d",&arr[i]);
    }
    for(i=0;i<5;i++)
    {
        sum = sum + arr[i];
    }
    printf("sum of all elements of an array is %d ",sum);
}
```

```
Enter the elements in an Array
1
2
3
4
5
sum of all elements of an array is 15
```

Exercise

Write a Program

1. To delete an element from a one-dimensional array at a given index.
2. To find the sum and average of elements in a one-dimensional array entered by the user.
3. To count the number of even and odd elements in an array.
4. To find the largest and smallest elements in an array.
5. To merge two sorted arrays into a single sorted array.
6. To create a menu driven program that
   - takes a decimal number as input and converts it to its binary equivalent.
   - takes a binary number as input and converts it to its decimal equivalent and then user should ask again to convert the decimal into octal decimal or hexadecimal number systems.
   - Hint: use array to store bits

SHARDA UNIVERSITY
Beyond Boundaries

2D Array initialization and access elements

int arr[3][3];

data type name of array rows column

```
int arr[3][3]={{25,10,5},{4,6,13},{45,90,78}};

int arr[3][3] = {0};
```

columns → / rows ↓	col 0	col 1	col 2
row 0	[0][0]	[0][1]	[0][2]
row 1	[1][0]	[1][1]	[1][2]
row 2	[2][0]	[2][1]	[2][2]

SHARDA UNIVERSITY
Beyond Boundaries

2D Array initialization and access elements

```c
#include<stdio.h>
main()
{
    int i,j, arr[5][5];

    printf("Enter the elements in 2d array\n");
    for(i=0;i<2;i++)
    {
        for(j=0;j<3;j++)
        {
            scanf("%d",&arr[i][j]);
        }
    }

    printf("Entered elements are: \n");
    for(i=0;i<5;i++)
    {
        for(j=0;j<5;j++)
        {
            printf("%d  ",arr[i][j]);
        }
        printf("\n");
    }
}
```

columns → / rows ↓	col 0	col 1	col 2	col3	col4
row 0	[0][0]	[0][1]	[0][2]	[0][3]	[0][4]
row 1	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]
row 2	[2][0]	[2][1]	[2][2]	[2][3]	[2][4]
row3	[3][0]	[3][1]	[3][2]	[3][3]	[3][4]
row4	[4][0]	[4][1]	[4][2]	[4][3]	[4][4]

SHARDA UNIVERSITY
Beyond Boundaries

2D Array initialization and access elements

```c
#include<stdio.h>
main()
{
    int i,j, arr[5][5];

    printf("Enter the elements in 2d array\n");
    for(i=0;i<2;i++)
    {
        for(j=0;j<3;j++)
        {
            scanf("%d",&arr[i][j]);
        }
    }

    printf("Entered elements are: \n");
    for(i=0;i<5;i++)
    {
        for(j=0;j<5;j++)
        {
            printf("%d  ",arr[i][j]);
        }
        printf("\n");
    }
}
```

```
Enter the elements in 2d array
1
2
3
4
5
6
Entered elements are:
1  2  3  1999404093  8
4  5  6  4199040  4199040
0  4200896  6422192  6422248  6422476
1999364464  622003018  -2  6422248  1999340141
4200896  6422404  4200990  4200896  3973120
```

Note: size of array is 5 x 5 and values entered in 2 X 3 size. Therefore, garbage value stored in unused locations.

Program to sum two matrices

```c
#include<stdio.h>
main()
{
    int i,j, m1[3][3], m2[3][3], m3[3][3];
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("Enter the 1st Matrix m1[%d][%d]:",i,j);
            scanf("%d",&m1[i][j]);
        }
    }
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("Enter the 2nd Matrix m2[%d][%d]:",i,j);
            scanf("%d",&m2[i][j]);
        }
    }
    // sum of 2 matrices
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            m3[i][j] = m1[i][j] + m2[i][j];
        }
        printf("\n");
    }
    printf("sum of 2 matrices is \n");
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("%d ",m3[i][j]);
        }
        printf("\n");
    }
}
```

```
Enter the 1st Matrix m1[0][0]:1
Enter the 1st Matrix m1[0][1]:2
Enter the 1st Matrix m1[0][2]:3
Enter the 1st Matrix m1[1][0]:4
Enter the 1st Matrix m1[1][1]:5
Enter the 1st Matrix m1[1][2]:6
Enter the 1st Matrix m1[2][0]:7
Enter the 1st Matrix m1[2][1]:8
Enter the 1st Matrix m1[2][2]:9
```

```
Enter the 2nd Matrix m2[0][0]:1
Enter the 2nd Matrix m2[0][1]:2
Enter the 2nd Matrix m2[0][2]:3
Enter the 2nd Matrix m2[1][0]:4
Enter the 2nd Matrix m2[1][1]:5
Enter the 2nd Matrix m2[1][2]:6
Enter the 2nd Matrix m2[2][0]:7
Enter the 2nd Matrix m2[2][1]:8
Enter the 2nd Matrix m2[2][2]:9
```

```
sum of 2 matrices is
2 4 6
8 10 12
14 16 18
```

SHARDA UNIVERSITY
Beyond Boundaries

Character Array declaration

char ch[5];

data type **name of array** **size of array**

```
char ch = 'A';
```

A

```
char ch[5] = {'A', 'B', 'C', 'D', 'E'};
```

A	B	C	D	E
0	1	2	3	4

Character Arrays

Character Array declaration

```c
#include<stdio.h>
main()
{
    int i;
    char ch[5]={'A','B','C','D','E'};
    for(i=0;i<5;i++)
    {
        printf("Character %c is at position %d\n",ch[i],&ch[i]);
    }

}
```

```
Character A is at position 6422263
Character B is at position 6422264
Character C is at position 6422265
Character D is at position 6422266
Character E is at position 6422267
```

SHARDA UNIVERSITY
Beyond Boundaries

Character Array declaration

```c
#include<stdio.h>
main()
{
    int i,j;
    char ch[]={'C','O','M','P','U','T','E','R'};
    for(i=0;i<=8;i++)
    {
        for(j=0;j<i;j++)
        {
            printf("%c ",ch[j]);
        }
        printf("\n");
    }
}
```

SHARDA UNIVERSITY
Beyond Boundaries

Character Array declaration

```c
#include<stdio.h>
main()
{
    int i,j;
    char ch[]={'C','O','M','P','U','T','E','R'};
    for(i=0;i<=8;i++)
    {
        for(j=0;j<i;j++)
        {
            printf("%c ",ch[j]);
        }
        printf("\n");
    }
    for(i=0;i<=8;i++)
    {
        for(j=0;j<i;j++)
        {
            printf("%d ",ch[j]);
        }
        printf("\n");
    }
}
```

```
C
C O
C O M
C O M P
C O M P U
C O M P U T
C O M P U T E
C O M P U T E R
```

Printing ASCII value of each character

```
67
67 79
67 79 77
67 79 77 80
67 79 77 80 85
67 79 77 80 85 84
67 79 77 80 85 84 69
67 79 77 80 85 84 69 82
```

Searching Algorithms

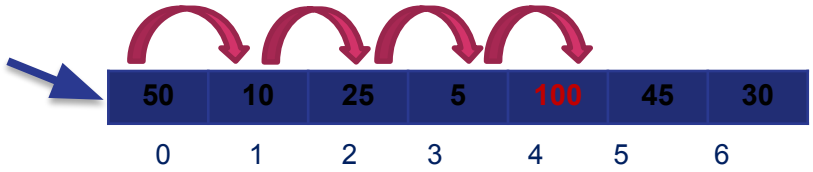
These algorithms are the fundamental concepts of Computer Science used to search or retrieve information stored within data structure.

1. **Linear or Sequential Search**
2. **Binary Search**

Linear or Sequential Search

Every single element from first position to last position within the array or linked list is checked with the **searching element,** if match is found it returns the location else NULL value returned.

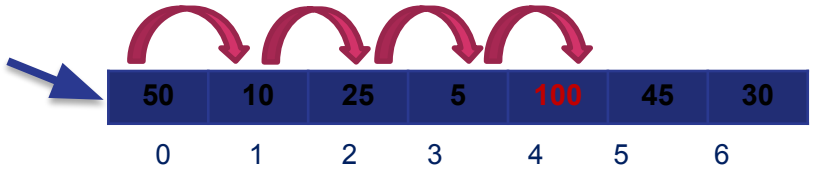
Element to be searched **100**



Linear or Sequential Search

Every single element from first position to last position within the array or linked list is checked with the **searching element,** if match is found it returns the location else NULL value returned.

Element to be searched **100**



Pseudo Code of Linear Search

```
Start
LinearSearch (Array , value)
For each element in the array
   If (searched element == value)
      Return the searched element  location
   end if
end for
End
```

Linear or Sequential Search

Every single element from first position to last position within the array or linked list is checked with the **searching element,** if match is found it returns the location else NULL value returned.

```c
#include<stdio.h>
main()
{
    int i, arr[10], value, flag=0;
    printf("Enter the elements:\n");
    for(i=0;i<10;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\nEnter the element to be searched within the array: ");
    scanf("%d", &value);

    for(i=0;i<10;i++)
    {
        if(arr[i]==value)
        {
            printf("\n\nElement %d is found at position %d", arr[i], i+1);
            flag = 1;
            break;
        }
    }
    if(flag==0)
    printf("\n\n%d is not found in an given array",value);
}
```

Element to be searched **36**

35	65	12	5	10	100	25	36	56	15

Linear or Sequential Search

Every single element from first position to last position within the array or linked list is checked with the **searching element,** if match is found it returns the location else NULL value returned.

```c
#include<stdio.h>
main()
{
    int i, arr[10], value, flag=0;
    printf("Enter the elements:\n");
    for(i=0;i<10;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\nEnter the element to be searched within the array: ");
    scanf("%d", &value);

    for(i=0;i<10;i++)
    {
        if(arr[i]==value)
        {
            printf("\n\nElement %d is found at position %d", arr[i], i+1);
            flag = 1;
            break;
        }
    }
    if(flag==0)
    printf("\n\n%d is not found in an given array",value);
}
```

```
Enter the elements:
35
65
12
5
10
100
25
36
56
15

Enter the element to be searched within the array: 36


Element 36 is found at position 8
```

Binary Search

- used to search an element in a sorted array.
- if array is not sorted order, then it must be sorted first in **ascending order**.
- works on the principle of divide and conquer.

10	20	30	40	50	60	70	80	90	95
0	1	2	3	4	5	6	7	8	9

Binary Search

- used to search an element in a sorted array.
- if array is not sorted order, then it must be sorted first in **ascending order**.
- works on the principle of divide and conquer.

10	20	30	40	50	60	70	80	90	95
0	1	2	3	4	5	6	7	8	9

Approach for Binary Search

- Find the middle element.
- Compare the target element with the middle element of the array.
- If the target element is greater than the middle element, then the search continues in the right half.
- Else if the target element is less than the middle value, the search continues in the left half.
- This process is repeated until the middle element is equal to the target element, or the target element is not in the array
- If the target element is found, its index is returned, else -1 is returned.

Binary Search

```c
#include<stdio.h>
main()
{
    int i, arr[10], n, value, mid, low, high ;
    printf("Enter the number of elements in ascending order: ");
    scanf("%d", &n);
    printf("\nEnter the %d elements\n",n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\nEnter the element to be searched within the array: ");
    scanf("%d", &value);
    low = 0;
    high = n - 1;
    mid = (low+high)/2;
    for(i=low;i<=high;i++)
    {
        if(arr[mid] < value)
        {
            low = mid + 1;
        }
        else if(arr[mid] == value)
        {
            printf("\n%d found at location %d",value, mid+1);
            break;
        }
        else
        {
            high = mid - 1;
        }
        mid = (low + high)/2;
    }
    if(low > high)
    printf("%d not found in the list", value);
}
```

Binary Search

```c
#include<stdio.h>
main()
{
    int i, arr[10], n, value, mid, low, high ;
    printf("Enter the number of elements in ascending order: ");
    scanf("%d", &n);
    printf("\nEnter the %d elements\n",n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\nEnter the element to be searched within the array: ");
    scanf("%d", &value);
    low = 0;
    high = n - 1;
    mid = (low+high)/2;
    for(i=low;i<=high;i++)
    {
        if(arr[mid] < value)
        {
            low = mid + 1;
        }
        else if(arr[mid] == value)
        {
            printf("\n%d found at location %d",value, mid+1);
            break;
        }
        else
        {
            high = mid - 1;
        }
        mid = (low + high)/2;
    }
    if(low > high)
    printf("%d not found in the list", value);
}
```

```
Enter the number of elements in ascending order: 7

Enter the 7 elements
10
20
25
30
36
70
85

Enter the element to be searched within the array: 70

70 found at location 6
```

Sorting Algorithms

Used to arrange the elements of an array either in ascending order or descending order.

Unsorted array

50	10	30	60	20	90	80	70	95	40
0	1	2	3	4	5	6	7	8	9

Sorted array

10	20	30	40	50	60	70	80	90	95
0	1	2	3	4	5	6	7	8	9

There are 5 basic Sorting Algorithms:

- **Insertion Sort**
- **Selection Sort**
- **Bubble Sort**
- Merge Sort
- Quick Sort

Sorting Algorithms

Used to arrange the elements of an array either in ascending order or descending order.

Unsorted array

50	10	30	60	20	90	80	70	95	40
0	1	2	3	4	5	6	7	8	9

Sorted array

10	20	30	40	50	60	70	80	90	95
0	1	2	3	4	5	6	7	8	9

There are 5 basic Sorting Algorithms:

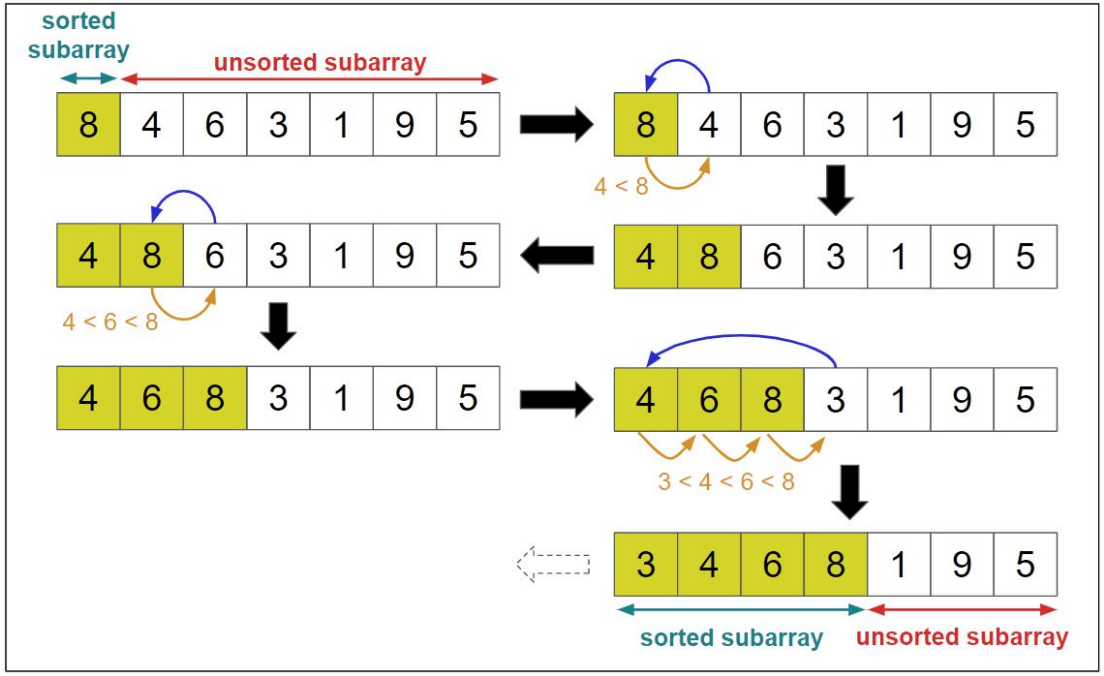
- **Insertion Sort**
- **Bubble Sort**
- **Selection Sort**
- Merge Sort
- Quick Sort

Insertion Sort

The principle behind insertion sort is to take one element, iterate through the sorted array & find its correct position in the sorted array.
Insertion Sort works in a similar manner as deck of cards are arranged.

Insertion Sort

The principle behind insertion sort is to take one element, iterate through the sorted array & find its correct position in the sorted array.
Insertion Sort works in a similar manner as deck of cards are arranged.



Insertion Sort

10	20	50	40	5	gb	gb	gb	gb	gb	gb	gb	gb	gb	gb
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

```c
#include<stdio.h>
main()
{
    int i, j, count, temp, arr[15];

   printf("Total number of elements to be entered in an array:\n");
   scanf("%d",&count);
   printf("\nEnter %d elements\n",count);
   for(i=0;i<count;i++)
   {
    scanf("%d",&arr[i]);

   }
   for(i=1;i<count;i++)
   {
      temp=arr[i];
      j=i-1;
      while((temp<arr[j])&&(j>=0))
      {
         arr[j+1]=arr[j];
         j=j-1;
      }
      arr[j+1]=temp;
   }
   printf("\n\nSorted elements are: ");
   for(i=0;i<count;i++)
      printf(" %d",arr[i]);
}
```

Insertion Sort

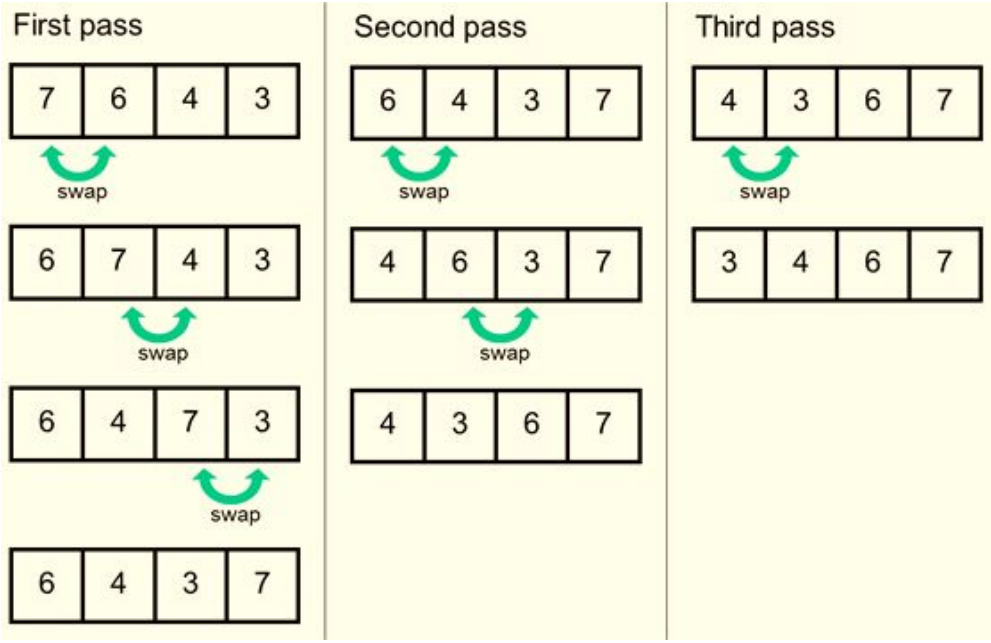
```c
#include<stdio.h>
main()
{
    int i, j, count, temp, arr[15];

   printf("Total number of elements to be entered in an array:\n");
   scanf("%d",&count);
   printf("\nEnter %d elements\n",count);
   for(i=0;i<count;i++)
   {
    scanf("%d",&arr[i]);

   }
   for(i=1;i<count;i++)
   {
      temp=arr[i];
      j=i-1;
      while((temp<arr[j])&&(j>=0))
      {
         arr[j+1]=arr[j];
         j=j-1;
      }
      arr[j+1]=temp;
   }
   printf("\n\nSorted elements are: ");
   for(i=0;i<count;i++)
      printf(" %d",arr[i]);
}
```

```
Total number of elements to be entered in an array:
10

Enter 10 elements
25
1
2
15
11
12
30
28
50
45


Sorted elements are:  1 2 11 12 15 25 28 30 45 50
```

Bubble Sort

It's a simplest algo., that checks and swap the elements if they are unordered.

1. Starting from the first index, compare the first and the second elements.
2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.
4. The above process goes on until the last element is not sorted.



Bubble Sort

```c
#include<stdio.h>
main()
{
    int arr[5],i,j,swap,k;
    printf("\nEnter 5 elements\n");
    for (i=0;i<5;i++)
    {
        scanf("%d", &arr[i]);
    }
    for(i=0;i<5-1;i++)
    {
        printf("\n______________");
        printf("\nPass %d",i+1);

        for(j=0;j<5-1-i;j++)
        {
            printf("\n\ni = %d, j = %d",i,j);
            if(arr[j+1] < arr[j])
            {
                swap=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=swap;
            }
            printf("\nUpdated array is\n");
            for(k=0;k<5;k++)
            {
                printf("%d|",arr[k]);
            }
        }
        printf("\n______________");
    }
    printf("\n\nSorted Array:");
    for(i=0;i<5;i++)
    {
        printf(" %d ",arr[i]);
    }
}
```

20	10	50	40	5
0	1	2	3	4

```
--------------
Pass 1

i = 0, j = 0
Updated array is
10|20|50|40|5|

i = 0, j = 1
Updated array is
10|20|50|40|5|

i = 0, j = 2
Updated array is
10|20|40|50|5|

i = 0, j = 3
Updated array is
10|20|40|5|50|
--------------
```

```
--------------
Pass 2

i = 1, j = 0
Updated array is
10|20|40|5|50|

i = 1, j = 1
Updated array is
10|20|40|5|50|

i = 1, j = 2
Updated array is
10|20|5|40|50|
--------------
```

```
--------------
Pass 3

i = 2, j = 0
Updated array is
10|20|5|40|50|

i = 2, j = 1
Updated array is
10|5|20|40|50|
--------------
```

```
--------------
Pass 4

i = 3, j = 0
Updated array is
5|10|20|40|50|
--------------
```

Exercise

Write a Program

1. To read two 2×2 matrices from the user and display their sum.
2. To find the transpose of a given matrix.
3. To find the determinant of a 3×3 matrix.
4. To perform linear search for an element in a one-dimensional array.
5. To implement Selection Sort to sort an array in descending order.
6. To implement Insertion Sort to sort an array in ascending order.
7. To sort each row of a matrix individually using any sorting technique.
8. Given an integer array of size N, the task is to find the count of elements whose value is greater than all of its prior elements.
9. A parking lot in a mall has RxC number of parking spaces. Each parking space will either be empty (0) or full (1). The status (0/1) of a parking space is represented as the element of the matrix. Write a program to find index of the row(R) in the parking lot that has the most of the parking spaces full (1).
10. An event has been organized in an organization. The event is organized for a limited time(T). The number of guests entering (E[i]) and leaving (L[i]), the event at every hour is represented as elements of the array. The task is to find the maximum number of guests present in the organization at any given instance within T hours.